

Plan de tests — Spec V2 (BR18-BR31)

Document de recette fonctionnelle, à dérouler **à la fin de chaque phase**, avant de démarrer la phase suivante. Référentiel : [docs/01-specifications/BMAD.md](#) (section 6.5, BR18-BR31), [docs/04-features/FEATURE_SPECIFICATIONS.md](#) (section 9), [docs/05-api/API_REFERENCE.md](#) (section 13). Environnement de test : `node serve.js` (port 3000) + backend `npm run backend:dev` (port 3001) connecté à Supabase (pas de mode local-only). Convention : chaque test précise **Entrée** → **Action** → **Résultat attendu**. Les tests "Persistance API" doivent être vérifiés par un **rechargement complet de la page** (F5) après l'action, pas seulement par l'état JS en mémoire.

Phase 1A — Second RIB (BR22) + Piste d'audit (BR23)

A. Paramètres — Second RIB (BR22)

- ☐ **Affichage des champs RIB n°1/RIB n°2** : ouvrir [Paramètres](#) → [Paramètres de facturation](#) → vérifier la présence des champs `company_rib/companyRibLabel` (RIB n°1, déjà existant) et `company_rib_2/companyRib2Label` (RIB n°2, nouveau), tous initialement vides ou pré-remplis depuis Supabase.
- ☐ **Enregistrement RIB n°1 seul** : laisser RIB n°2 vide, saisir un RIB n°1 + libellé, cliquer "Enregistrer" → recharger la page → les valeurs RIB n°1 sont restaurées depuis l'API (`PUT /settings` puis re-`GET /settings` au chargement), RIB n°2 reste vide.
- ☐ **Enregistrement des deux RIB** : saisir RIB n°1 (avec libellé) ET RIB n°2 (avec libellé "RIB 2 - Zaghouan" par ex.) → "Enregistrer" → recharger la page → les 4 champs (`company_rib`, `company_rib_label`, `company_rib_2`, `company_rib_2_label`) sont bien persistés et réaffichés.
- ☐ **RIB n°2 sans libellé** : saisir uniquement `company_rib_2` (valeur RIB) sans libellé → enregistrer → vérifier que l'absence de libellé n'empêche pas l'enregistrement et que le sélecteur de facture (cf. ci-dessous) affiche alors un libellé par défaut générique (ex. "RIB n°2") plutôt qu'un libellé vide ou `undefined`.
- ☐ **Suppression du RIB n°2** : avec un RIB n°2 déjà enregistré, vider les champs `company_rib_2/company_rib_2_label` → enregistrer → recharger → les champs sont bien vides en base (pas de chaîne résiduelle), et le sélecteur RIB du formulaire facture redevient masqué (cf. test ci-dessous).
- ☐ **Persistance via /demo/reset** : cliquer "Charger données démo" → vérifier que les deux RIB du jeu de données démo (s'ils sont prévus) sont bien renseignés en base après reset, et rechargés correctement à l'écran Paramètres.

B. Sélecteur de RIB sur le formulaire facture (BR22)

- ☐ **Sélecteur masqué si RIB n°2 absent** : avec uniquement RIB n°1 configuré, ouvrir le formulaire "Créer facture" → le sélecteur "RIB" n'apparaît pas (ou est masqué), le RIB n°1 est utilisé implicitement.
- ☐ **Sélecteur visible si RIB n°2 configuré** : configurer un RIB n°2 → ouvrir "Créer facture" → un sélecteur "RIB" apparaît avec deux options : "RIB n°1 — {libellé ou 'RIB n°1'}" et "RIB n°2 — {libellé ou 'RIB n°2'}", **pré-sélectionné sur RIB n°1**.
- ☐ **Création facture avec RIB n°1 (valeur par défaut)** : créer une facture sans changer le sélecteur → enregistrer (`POST /invoices`) → vérifier en base que `invoices.rib` = valeur de `company_rib` et

`invoices.rib_label = company_rib_label` (ou libellé par défaut), figés sur la facture.

- ☐ **Création facture avec RIB n°2** : sélectionner explicitement "RIB n°2" → enregistrer → vérifier `invoices.rib = company_rib_2` et `invoices.rib_label = company_rib_2_label`.
- ☐ **PDF facture — RIB n°1** : générer le PDF d'une facture créée avec RIB n°1 → le bloc coordonnées bancaires du PDF affiche le RIB n°1 et son libellé.
- ☐ **PDF facture — RIB n°2** : générer le PDF d'une facture créée avec RIB n°2 → le PDF affiche bien le RIB n°2 (différent du RIB n°1), pas le RIB n°1 par défaut.
- ☐ **Non-régression — modification ultérieure des paramètres** : après création d'une facture avec RIB n°2, modifier/supprimer `company_rib_2` dans Paramètres → régénérer le PDF de cette même facture (déjà créée) → le RIB affiché reste celui figé au moment de la création (`invoices.rib/rib_label`), inchangé malgré la modification des paramètres.
- ☐ **Facture existante (pré-V2) sans `rib/rib_label`** : ouvrir une facture créée avant l'implémentation BR22 (`rib/rib_label` NULL) → vérifier que le PDF ne plante pas et affiche un repli cohérent (RIB n°1 courant des paramètres, ou mention vide propre — pas "undefined"/"null").
- ☐ **Persistance API stricte** : créer une facture avec RIB n°2, recharger la page entière (F5), rouvrir cette facture via l'éditeur générique → `rib/ribLabel` sont bien rechargés depuis l'API (pas seulement présents en mémoire avant rechargement).

C. Piste d'audit `created_by/updated_by/created_at/updated_at` (BR23)

- ☐ **Création — `created_by` renseigné** : connecté en tant qu'utilisateur A, créer un nouveau client (POST `/customers`) → vérifier en base que `created_by = A.id`, `updated_by = A.id` (ou NULL selon convention retenue — à clarifier, cf. section "Points d'attention"), `created_at/updated_at` horodatés à l'instant de création.
- ☐ **Modification — `updated_by` mis à jour, `created_by` préservé** : connecté en tant qu'utilisateur B, modifier ce même client via l'éditeur générique (PUT `/customers/:id`) → vérifier que `updated_by = B.id` et `updated_at` change, mais que `created_by = A.id` et `created_at` restent inchangés.
- ☐ **Couverture multi-entités** : répéter le test création + modification (au minimum création, idéalement +modification) pour chacune des entités suivantes, en vérifiant `created_by/updated_by/created_at/updated_at` en base via l'API : `cars`, `customers`, `contracts`, `invoices`, `payments`, `reservations`, `maintenanceCosts`, `insurances`, `insuranceInstallments`, `leasingContracts`, `leasingInstallments`, `vignettes`, `inspections`, `settings`.
- ☐ **Affichage dans l'éditeur générique — lecture seule** : ouvrir l'éditeur générique (`openRecordEditor`) sur un client → vérifier la présence des 4 champs `created_by`, `updated_by`, `created_at`, `updated_at`, tous **désactivés/non modifiables** (`disabled/readOnly`), avec le style "champ calculé" déjà utilisé pour les autres champs en lecture seule.
- ☐ **Résolution du nom d'utilisateur (pas l'ID brut)** : pour un enregistrement dont `created_by/updated_by` correspondent à des ids existants dans `state.users`, vérifier que l'éditeur affiche le **nom** de l'utilisateur (ex. "Mehdi Souihli") et non l'UUID/identifiant brut.
- ☐ **Résolution avec utilisateur inconnu/supprimé** : pour un enregistrement dont `created_by/updated_by` référencent un id absent de `state.users` (utilisateur supprimé) → l'éditeur n'affiche pas une erreur JS, affiche un repli explicite (ex. "Utilisateur inconnu ({id})" ou "—") plutôt qu'`undefined`.
- ☐ **Enregistrement sans modification des champs audit** : dans l'éditeur générique, modifier un champ métier (ex. téléphone du client) et enregistrer → vérifier que les champs `created_by/created_at` (désactivés, non envoyés dans le payload PUT) ne sont **pas écrasés** côté backend — `created_by/created_at` restent ceux d'origine après l'enregistrement.

- ☐ **Tri par défaut `created_at` décroissant** : ouvrir chaque écran liste impacté par BR23 (Clients, Voitures, Contrats, Factures, Paiements, Réservations, Maintenance, Assurances, Leasing, Vignettes) → vérifier que les enregistrements sont triés par `created_at` décroissant (le plus récent en premier) à l'ouverture, **avant** toute action de tri manuel (BR24 sera implémenté en Phase 1B, mais le tri par défaut BR23 doit déjà être effectif).
- ☐ **Enregistrements existants (pré-V2) sans `created_by/updated_by`** : pour un enregistrement créé avant la migration (valeurs `created_by/updated_by` NULL), vérifier que l'éditeur générique affiche ces champs sans erreur (ex. "—" ou "Non renseigné"), et que le tri par `created_at` décroissant gère correctement les valeurs NULL (placées en fin de liste, ou `created_at` rétro-rempli par la migration — à confirmer).

D. Contrainte `vat_rate >= 0` (rattaché à BR18, déjà en partie présent)

- ☐ **Formulaire Paramètres — TVA négative rejetée côté UI** : dans `Paramètres` → `Paramètres de facturation`, saisir une valeur négative dans le champ "TVA (%)" → enregistrer → message d'erreur explicite affiché (pas de `PUT /settings` envoyé, ou réponse 400 gérée proprement), `state.settings.vatRate` n'est pas modifié.
- ☐ **Backend — TVA négative rejetée (`PUT /settings`)** : appeler directement `PUT /settings` avec `vatRate: -5` (en contournant l'UI, ex. via un client API) → réponse 400 avec message exploitable, `settings.vat_rate` en base reste inchangé.
- ☐ **Backend — contrainte `chk_vat_rate_non_negative`** : tenter une écriture SQL directe (ou via PostgREST sans passer par la validation applicative) avec `vat_rate = -1` → la contrainte `CHECK (vat_rate >= 0)` rejette l'écriture (erreur 23514), le backend traduit cette erreur en réponse propre (pas un 500 brut).
- ☐ **TVA = 0 acceptée** : saisir `vatRate = 0` → enregistrement accepté (0 est une valeur valide, différente de "négative").
- ☐ **Non-régression — calcul HT↔TTC facture avec TVA modifiée** : modifier `vatRate` (valeur positive différente, ex. 19 → 7) → créer une nouvelle facture → vérifier que `vat_amount/amount_tnd` recalculés utilisent le nouveau taux ; les factures existantes (déjà émises) ne sont pas recalculées rétroactivement.

E. Non-régression Phase 1A

- ☐ **Connexion / multi-utilisateur** : se connecter avec deux comptes utilisateurs différents (de deux agences différentes si applicable) → vérifier que le chargement des données (`loadDataFromAPI`) fonctionne toujours normalement, et que les nouveaux champs audit n'empêchent pas l'affichage des grilles existantes.
- ☐ **Création/édition/suppression sur chaque entité impactée** : pour `cars`, `customers`, `contracts`, `invoices`, `payments`, `reservations`, `maintenanceCosts`, `insurances`, `leasingContracts`, `vignettes` : créer un enregistrement, le modifier via l'éditeur générique, le supprimer → aucune régression fonctionnelle (les routes modifiées pour `stampCreate/stampUpdate` ne cassent pas les payloads existants).
- ☐ **/demo/reset après modification du schéma** : cliquer "Charger données démo" → vérifier que la réinitialisation fonctionne malgré les nouvelles colonnes `created_by/updated_by/company_rib_2*` (pas d'erreur 500 due à une contrainte `NOT NULL` ou `FK users.id` non satisfaite par le jeu de données démo).
- ☐ **Dashboard et graphiques** : ouvrir le dashboard → vérifier que les KPI et graphiques cliquables (commit `b533164`) fonctionnent toujours (les nouvelles colonnes ne doivent pas modifier les agrégats

existants).

Phase 1B — Tri et filtre génériques (BR24)

Reportée après la Phase 1A. À tester sur chaque grille listée dans la spec : Contrats, Factures, Voitures, Clients, Réservations, Paiements, Maintenance, Assurances, Leasing, Vignettes, et — une fois disponibles — Lignes de contrat/facture/devis (Phases 2-4).

A. Tri générique

- ☐ **Tri ascendant au premier clic** : sur l'écran Voitures, cliquer sur l'en-tête de colonne "Plaque" → la grille se trie par plaque en ordre croissant, un indicateur visuel ▲ apparaît sur la colonne.
- ☐ **Tri descendant au second clic** : cliquer une seconde fois sur "Plaque" → tri inversé (décroissant), indicateur ▼.
- ☐ **Changement de colonne de tri** : trier sur "Plaque", puis cliquer sur "Statut" → le tri bascule sur "Statut" (ascendant), l'indicateur disparaît de "Plaque" et apparaît sur "Statut".
- ☐ **Tri sur colonne numérique** : sur l'écran Factures, trier par "Total TTC" → vérifier un tri numérique correct (10 < 100, pas un tri alphabétique "10" > "2").
- ☐ **Tri sur colonne date** : trier par `created_at` (ou date du contrat) → ordre chronologique correct.
- ☐ **Tri par défaut `created_at` décroissant (cohérence BR23)** : ouvrir un écran sans interaction → vérifier que le tri par défaut est bien `created_at` décroissant (déjà couvert en 1A, à re-vérifier ici car BR24 généralise l'implémentation via `renderSortableFilterableTable`).

B. Filtre générique

- ☐ **Filtre texte — recherche partielle, insensible à la casse** : sur l'écran Clients, dans le filtre de la colonne "Nom", saisir "mart" → seules les lignes contenant "mart"/"Mart"/"MART" (ex. "Martin", "Smart Rental") restent affichées.
- ☐ **Filtre énuméré (liste déroulante)** : sur l'écran Voitures, filtrer la colonne "Statut" via une liste déroulante (`dispo/loue/maintenance`) → seules les voitures du statut sélectionné apparaissent.
- ☐ **Filtres cumulatifs (ET logique)** : sur l'écran Contrats, filtrer simultanément `Statut = actif` ET `Client` contenant "Ben" → seules les lignes respectant les deux critères apparaissent.
- ☐ **Réinitialisation du filtre** : vider le champ de filtre texte → la grille réaffiche toutes les lignes.
- ☐ **Filtre + tri combinés** : appliquer un filtre puis trier sur une colonne → le tri s'applique uniquement aux lignes filtrées (le filtre reste actif après le tri).
- ☐ **Filtre sur grille vide** : appliquer un filtre qui ne correspond à aucune ligne → affichage d'un message "Aucun résultat" (pas de tableau vide silencieux ni d'erreur JS).

C. État de tri/filtre en session

- ☐ **Persistance en mémoire pendant la session** : appliquer un tri/filtre sur l'écran Factures, naviguer vers un autre onglet (ex. Dashboard), puis revenir sur Factures → le tri/filtre appliqué précédemment est conservé (`state.ui.tableState`).
- ☐ **Réinitialisation au rechargement de page** : appliquer un tri/filtre, recharger la page (F5) → le tri repasse par défaut à `created_at` décroissant et les filtres sont vidés (pas de persistance obligatoire en base, conformément à la spec).

- ☐ **Indépendance entre écrans** : appliquer un tri sur "Voitures" (ex. par plaque) et un filtre sur "Clients" (ex. par nom) → vérifier que chaque écran conserve son propre état indépendamment (`tableState[entityKey]`).

D. Navigation cliquable et widgets (impact de BR24)

- ☐ **Navigation widget → grille filtrée, compatible avec le filtre générique** : cliquer sur le KPI "Contrats actifs" du dashboard → bascule vers `#contracts` avec le filtre `status=actif` **pré-rempli dans la nouvelle ligne de filtres générique** (pas un filtre caché ou une variable JS invisible à l'utilisateur) → la grille affiche bien uniquement les contrats actifs, et le champ de filtre "Statut" affiche visuellement la valeur "actif".
- ☐ **Cohérence filtre pré-rempli + tri par défaut** : après une navigation filtrée depuis un widget, vérifier que le tri par défaut (`created_at` décroissant) reste appliqué en plus du filtre.

E. Non-régression Phase 1B

- ☐ **Toutes les grilles existantes restent fonctionnelles** : pour chaque écran (Voitures, Clients, Contrats, Factures, Paiements, Réservations, Maintenance, Assurances, Leasing, Vignettes), vérifier que double-clic sur une ligne ouvre toujours l'éditeur générique (`openRecordEditor`) après l'introduction du tri/filtre (les gestionnaires d'événements `dblclick` ne sont pas cassés par le nouveau rendu de tableau).
- ☐ **Performance sur volume réaliste** : charger un jeu de données avec un nombre significatif d'enregistrements (ex. 200+ contrats) → tri/filtre restent réactifs côté client (pas de blocage UI), conformément à l'implémentation "tout en mémoire" actée pour BR24.
- ☐ **Graphiques et KPI du dashboard** : vérifier que le rendu des graphiques Chart.js (commit `72887d3/b533164`) n'est pas affecté par le remplacement des fonctions `render<Entity>()` par `renderSortableFilterableTable`.

Phase 2 — Contrat entête + lignes (BR18, BR19, BR20, BR20bis, BR25, BR26)

A. Migration des contrats existants (BR20)

- ☐ **Contrat mono-véhicule existant migré** : après migration, un contrat existant (créé en V1, avec champs véhicule/période directement sur `contracts`) apparaît comme 1 entête `contracts` (sans champs véhicule propres) + exactement 1 `contract_lines` reprenant véhicule/période/tarif/statut d'origine.
- ☐ **Totaux entête après migration** : pour un contrat migré, `contracts.total_amount_ht/total_vat_amount/total_amount_ttc` = valeurs de la ligne unique migrée (somme à 1 ligne).
- ☐ **Aucune perte de données après migration** : comparer un échantillon de contrats avant/après migration (export ou requête directe) → tous les champs métier (client, dates, tarif, statut, notes) sont préservés.

B. Calcul HT ⇌ TTC sur ligne de contrat (BR18)

- ☐ **Saisie HT → calcul TTC** : sur le formulaire "+ Ajouter une ligne" d'un contrat, saisir `amount_ht = 1000` avec `vat_rate = 19` → `amount_ttc` se met à jour automatiquement à `1190` (sans taxe journalière ni timbre, à la différence de BR15bis).

- ☐ **Saisie TTC → calcul HT et rate** : saisir directement `amount_ttc = 1190` → `amount_ht` recalculé à `1000`, et `rate` (tarif HT/jour ou /mois) recalculé en conséquence selon la durée de la ligne.
- ☐ **rate comme champ pivot** : modifier `rate` (tarif HT/jour) puis modifier le nombre de jours de la période → `amount_ht = rate × jours` est recalculé, puis `amount_ttc` suit.
- ☐ **TVA = 0** : avec `vat_rate = 0`, saisir `amount_ht = 500` → `amount_ttc = 500` (pas de division par zéro ni d'erreur).
- ☐ **Valeurs limites — amount_ht = 0** : saisir `amount_ht = 0` → `amount_ttc = 0`, la ligne est acceptée (ou rejetée avec message clair si une règle de minimum existe — à clarifier dans la spec si besoin).
- ☐ **Valeur négative rejetée** : tenter de saisir `amount_ht = -100` → message d'erreur inline, enregistrement bloqué côté UI ET backend (`POST/PUT /contract-lines`).
- ☐ **Recalcul des totaux entête à chaque modification de ligne** : ajouter une 2e ligne à un contrat existant → `contracts.total_amount_ht/total_vat_amount/total_amount_ttc` = somme des deux lignes ; supprimer une ligne → totaux recalculés (somme de la ligne restante) ; modifier le montant d'une ligne → totaux recalculés immédiatement à l'écran ET persistés.

C. Contrôle de chevauchement véhicule (BR19)

- ☐ **Création — chevauchement détecté (UI)** : créer une ligne de contrat pour le véhicule X sur la période 10/06-20/06, alors qu'une autre ligne `active` pour X existe déjà sur 15/06-25/06 → message d'erreur rouge inline précisant véhicule/contrat/période en conflit, enregistrement bloqué.
- ☐ **Création — pas de chevauchement (périodes adjacentes)** : créer une ligne pour X sur 21/06-30/06 (le lendemain de la fin du contrat existant 10/06-20/06) → aucun conflit, enregistrement accepté (vérifier la borne : chevauchement strict vs. inclusif sur les dates de transition).
- ☐ **Modification — chevauchement introduit par modification de période** : modifier la période d'une ligne existante pour qu'elle chevauche désormais une autre ligne `active` du même véhicule → même contrôle déclenché, blocage.
- ☐ **Lignes brouillon exclues du contrôle** : créer une ligne `brouillon` pour le véhicule X sur une période qui chevauche une ligne `active` existante du même véhicule → **aucun blocage** (BR19 exclut explicitement les lignes `brouillon`).
- ☐ **Lignes annule/termine/resilie exclues du contrôle** : créer une ligne pour le véhicule X sur une période qui chevauche une ancienne ligne `resilie/termine/annule` du même véhicule → aucun blocage (statut exclu).
- ☐ **Chevauchement avec une réservation active (BR25)** : créer une ligne de contrat `active` pour le véhicule X sur une période qui chevauche une `reservation` confirmée existante (non liée à un contrat) pour X → conflit détecté et bloqué.
- ☐ **Backend — niveau 2, contournement UI** : appeler directement `POST /contract-lines` (sans passer par l'UI) avec un chevauchement → réponse `409 vehicle_overlap` avec le corps `{ success: false, error: "vehicle_overlap", message, conflict: { carId, contractId, lineId, periodStart, periodEnd } }`.
- ☐ **Backend — PUT /contract-lines/:id revalide BR19** : modifier via API directe `period_start/period_end/car_id` d'une ligne existante de façon à créer un chevauchement → `409 vehicle_overlap`.
- ☐ **Base de données — niveau 3 (contrainte EXCLUDE)** : simuler un accès concurrent (deux requêtes quasi simultanées créant des lignes en conflit pour le même véhicule, ex. deux onglets/process) → la seconde requête échoue avec `409 vehicle_overlap` traduit depuis l'erreur PostgreSQL `23P01`, pas un 500 brut.

- ☐ **Message d'erreur exploitable et localisé** : vérifier que le message retourné par le backend en cas de 409 est en français et directement affichable à l'utilisateur (pas un message technique brut de PostgreSQL).

D. Contrat entête + lignes, statut **brouillon** (BR20)

- ☐ **Création d'un contrat **brouillon**** : créer un contrat avec `status = brouillon` et 2 lignes → vérifier que les 2 `contract_lines` sont créées avec `status = brouillon`.
- ☐ **Lignes **brouillon** non engageantes** : avec un contrat **brouillon** ayant une ligne sur véhicule X/période P, créer un second contrat **actif** avec une ligne sur le même véhicule X/même période P → **aucun conflit BR19** (ligne brouillon exclue), et le planning de disponibilité du véhicule X ne montre pas la période P comme occupée par le contrat brouillon.
- ☐ **Confirmation **brouillon** → **actif sans conflit**** : sur un contrat **brouillon** sans conflit potentiel, cliquer "Confirmer le contrat" → `contracts.status = actif`, toutes les `contract_lines` passent à **active**, et pour chaque ligne, BR25 s'exécute (réservation créée/liée).
- ☐ **Confirmation **brouillon** → **actif avec conflit sur une ligne**** : sur un contrat **brouillon** ayant une ligne dont le véhicule/période chevauche désormais une ligne **active** d'un autre contrat (créée entre-temps) → la confirmation est bloquée **pour cette ligne** avec le message d'erreur rouge inline habituel ; le contrat reste **brouillon** (ou les autres lignes sans conflit sont-elles confirmées partiellement ? — à clarifier/vérifier le comportement transactionnel attendu, cf. BR20bis : rollback complet attendu).
- ☐ **Transition affichée **active** → **termine**** : pour une ligne `status = active` dont `period_end` est dans le passé → l'écran liste Contrats et le détail affichent le badge "termine" pour cette ligne, **sans écriture immédiate en base** (vérifier via requête API directe que `contract_lines.status` reste **active** juste après le simple affichage).
- ☐ **Persistance de la transition à la prochaine action** : sur cette même ligne affichée "termine" (mais encore **active** en base), effectuer une action (ex. modification d'un champ, ou résiliation) → `contract_lines.status` est mis à jour à **termine** en base à cette occasion.

E. Écran liste et détail Contrats (entête + lignes)

- ☐ **Écran liste — colonnes entête** : `#contracts` affiche bien Id, Client, Date, Type, Statut, Total HT, Total TTC, Nombre de véhicules (= `count(contract_lines)`), pour un contrat multi-lignes.
- ☐ **Double-clic → détail entête + lignes** : double-cliquer une ligne de la grille Contrats → ouverture d'un écran de détail avec un pavé entête (lecture seule pour les totaux) et un pavé lignes (`contract_lines`) avec colonnes Véhicule, Période, Jours/Mois, Tarif HT, Montant HT, TVA, Montant TTC, Statut, Actions.
- ☐ **Ajout d'une ligne depuis le détail** : depuis l'écran détail, "+ Ajouter une ligne" → sélection véhicule + période + tarif, contrôle de chevauchement en direct (C), calcul HT↔TTC (B) → la nouvelle ligne apparaît dans le pavé lignes et les totaux entête sont recalculés.
- ☐ **Suppression d'une ligne** : supprimer une `contract_line` (action "Supprimer la ligne") → la ligne disparaît, les totaux entête recalculés ; si la ligne avait une réservation créée par BR25 (et non préexistante), la réservation associée est supprimée (cf. F).
- ☐ **Cellule "Nombre de véhicules" cliquable (navigation widgets)** : cliquer sur la cellule "Nombre de véhicules" d'une ligne de la grille Contrats → ouvre le détail du contrat directement sur le pavé `contract_lines`.

F. Réservation liée à une ligne de contrat (BR25)

- ☐ **Aucune réservation préexistante → création** : créer une ligne **active** pour véhicule X / période P sans réservation préexistante pour X sur P → après création, **contract_lines.reservation_id** est renseigné, une nouvelle **reservations** existe avec **status = confirmée**, **start_date/end_date = P**, **start_time/end_time = 09:00/18:00**, **contract_line_id** = id de la ligne.
- ☐ **Réservation préexistante exacte → liaison sans duplication** : avec une réservation existante pour véhicule X exactement sur la période P (statut actif, non liée), créer une ligne de contrat active pour X/P → la réservation existante est liée (**reservations.contract_line_id** + **contract_lines.reservation_id**), **aucune nouvelle réservation créée** (vérifier le compte total de **reservations** avant/après).
- ☐ **Réservation préexistante à dates différentes → correction** : avec une réservation pour X sur 10/06-18/06, créer une ligne de contrat active pour X sur 10/06-20/06 (période différente mais chevauchante) → la réservation existante est corrigée (**start_date/end_date = 10/06-20/06**) plutôt que de créer un doublon.
- ☐ **Ordre des opérations respecté** : vérifier (via logs backend ou inspection de la transaction **create_contract_with_lines**) que la **contract_line** est insérée avant la recherche/création de réservation, et que **reservation_id** est mis à jour en dernier.
- ☐ **Suppression de ligne avant validation — réservation créée par BR25 supprimée** : créer un contrat **brouillon** avec une ligne, le confirmer (**brouillon** → **actif**, déclenchant BR25 et création d'une réservation), puis supprimer la ligne → la réservation créée par BR25 est supprimée.
- ☐ **Suppression de ligne — réservation préexistante NON supprimée** : même scénario, mais où la réservation liée préexistait avant la ligne de contrat (liée sans création) → après suppression de la ligne, cette réservation préexistante **n'est pas supprimée** (seulement déliée, **contract_line_id = NULL**).
- ☐ **Navigation cliquable — ligne résiliée → réservation associée** : pour une **contract_line** au statut **resilié**, cliquer sur la ligne (ou un lien dédié) → ouvre le détail de la réservation associée (**reservations.contract_line_id**).

G. Résiliation anticipée (BR26)

- ☐ **Résiliation standard, en avance** : sur une ligne **active** (période 01/06-30/06), bouton "Résilier" → saisir **actual_end_date = 15/06** (≥ aujourd'hui) → **contract_lines.status = resilié**, **actual_end_date = 15/06** ; **reservations.end_date** raccourcie à 15/06 (si **actual_end_date < period_end**).
- ☐ **Validation des bornes actual_end_date** : tenter **actual_end_date < period_start** → rejeté avec message clair ; tenter **actual_end_date > period_end** → rejeté avec message clair.
- ☐ **Résiliation rétroactive — confirmation explicite** : saisir **actual_end_date** dans le passé (< aujourd'hui) → une boîte de confirmation explicite est affichée avant validation ; après confirmation, **reservations.status = annulée** (en plus de la mise à jour de **contract_lines**).
- ☐ **Annulation de la confirmation rétroactive** : sur la boîte de confirmation rétroactive, cliquer "Annuler" → aucune modification n'est persistée (**contract_lines.status** reste **actif**).
- ☐ **Mise à jour du statut du contrat — dernière ligne active résiliée** : pour un contrat avec une seule ligne **active**, résilier cette ligne → **contracts.status** passe à **resilié**.
- ☐ **Mise à jour du statut du contrat — lignes actives restantes** : pour un contrat avec 2 lignes **active**, résilier une seule → **contracts.status** reste **actif**, seule la ligne résiliée change de statut.
- ☐ **Disponibilité du véhicule après résiliation** : après résiliation d'une ligne sur véhicule X avec **actual_end_date = 15/06** (période d'origine jusqu'au 30/06), créer une nouvelle ligne de contrat pour X sur 16/06-25/06 → **aucun conflit BR19** (le véhicule est disponible dès le lendemain de **actual_end_date**).

- ☐ **Recalcul HT/TVA/TTC au prorata** : pour une ligne `period_start = 01/06, period_end = 30/06` (30 jours, `amount_ht = 3000`), résiliée à `actual_end_date = 15/06` (15 jours) → `amount_ht` recalculé ≈ 1500 (prorata 15/30), `vat_amount/amount_ttc` recalculés en conséquence.
- ☐ **Répercussion sur la prochaine facture** : générer une facture pour ce contrat après résiliation → la ligne de facture générée automatiquement (BR21, Phase 3) reprend le montant HT **ajusté** (prorata), pas le montant d'origine de la ligne.

H. Atomicité (BR20bis) — `create_contract_with_lines`

- ☐ **Création réussie en une transaction** : créer un contrat avec 3 lignes valides (sans conflit) → vérifier qu'une seule requête `POST /rpc/create_contract_with_lines` est envoyée (pas 1 `POST /contracts` + 3 `POST /contract-lines` séparés), et que l'entête + les 3 lignes existent en base après l'appel.
- ☐ **Échec sur une ligne → rollback complet** : créer un contrat avec 3 lignes dont la 2e provoque un conflit BR19 → après l'échec, **aucune** entête `contracts` ni aucune `contract_lines` (y compris les 2 lignes valides) n'est créée en base — pas d'entête orpheline.
- ☐ **Pas d'état intermédiaire visible** : pendant/après un échec de création, vérifier qu'aucune réservation BR25 n'a été créée pour les lignes qui auraient été valides (tout ou rien, y compris les effets de bord BR25).
- ☐ **Mise à jour structurelle (ajout/suppression de ligne sur contrat existant)** : si l'ajout d'une ligne à un contrat existant utilise également une transaction — ajouter une ligne valide → succès ; ajouter une ligne en conflit → rollback, le contrat conserve son état (lignes) d'avant la tentative.

I. Persistance API (checklist transversale)

- ☐ **loadDataFromAPI charge contracts + contract_lines** : recharger la page → l'écran Contrats affiche les entêtes ET leurs lignes correctement, données issues de l'API (pas du seed démo local).
- ☐ **GET /contracts/:id/lines** : appeler cet endpoint pour un contrat multi-lignes → renvoie bien la liste complète des `contract_lines` de ce contrat.
- ☐ **syncStateToAPI inclut contract_lines** : après création de lignes en mémoire (si applicable à un flux de synchronisation), vérifier que `contract_lines` est bien transmis/synchronisé.
- ☐ **/demo/reset régénère des contrats avec lignes** : "Charger données démo" → les contrats du jeu démo sont bien structurés en entête + au moins 1 ligne chacun, totaux cohérents.

J. Non-régression Phase 2

- ☐ **Écran Réservations** : vérifier que l'écran Réservations affiche toujours correctement les réservations existantes (y compris celles créées/liées via BR25), et que `findReservationConflict` généralisé continue de fonctionner pour les réservations manuelles (hors contrat).
- ☐ **Planning/disponibilité véhicule** : l'écran de planning des véhicules reflète correctement les nouvelles réservations issues de BR25 et les lignes `brouillon` non engageantes (absentes du planning).
- ☐ **Dashboard KPI "Contrats actifs"** : le KPI compte désormais les **contrats** (entêtes) au statut `actif`, pas les lignes — vérifier la cohérence du chiffre affiché et son comportement cliquable (`#contracts` filtré `status=actif`).
- ☐ **Tri/filtre génériques (Phase 1B) sur Contrats et contract_lines** : si Phase 1B est déjà livrée, vérifier que le tri/filtre fonctionne aussi sur la nouvelle grille `contract_lines` du détail contrat.
- ☐ **Audit (Phase 1A) sur contract_lines** : `created_by/updated_by/created_at/updated_at` sont bien renseignés à la création/modification d'une `contract_line`, et affichés (résolus) dans l'éditeur

générique si `contract_lines` y est exposé.

Phase 3 — Facture entête + lignes (BR21, BR20bis)

A. Auto-remplissage depuis un contrat

- ☐ **Sélection d'un contrat → génération automatique des lignes** : sur le formulaire de création de facture, sélectionner un contrat ayant 2 `contract_lines` actives → 2 `invoice_lines` sont générées automatiquement, pré-remplies avec véhicule, immatriculation, période, `amount_ht` repris de chaque `contract_lines`.
- ☐ **Modification des lignes générées avant enregistrement** : modifier le `amount_ht` d'une ligne générée (ex. facturation partielle) → le calcul TVA/taxe journalière de cette ligne se recalcule (BR15bis), avant tout enregistrement.
- ☐ **Suppression d'une ligne générée** : supprimer une des lignes auto-générées avant enregistrement → la facture ne conserve que la ligne restante ; si c'était la dernière ligne, le bouton "Enregistrer" se désactive (cf. C).
- ☐ **Changement de contrat sélectionné** : sélectionner un contrat, puis changer pour un autre contrat avant enregistrement → les lignes générées pour le premier contrat sont remplacées par celles du second (pas d'accumulation).
- ☐ **Reprise d'une ligne résiliée au prorata (lien avec BR26)** : sélectionner un contrat dont une `contract_line` a été résiliée avec recalcul prorata (Phase 2.G) → la ligne de facture générée reprend le montant HT **ajusté** (prorata), pas le montant d'origine.

B. Calcul par ligne (BR15bis appliqué aux `invoice_lines`)

- ☐ **TVA et taxe journalière par ligne** : pour une facture à 2 lignes (véhicules différents, durées différentes : 10 jours et 5 jours), vérifier que `vat_amount` et `daily_tax_amount` sont calculés **indépendamment pour chaque ligne** ($TVA_{ligne} = HT_{ligne} \times \text{taux}/100$, $taxe_{ligne} = nb_jours_{ligne} \times \text{taxe_par_jour}$).
- ☐ **Timbre fiscal unique pour la facture** : pour cette même facture à 2 lignes, le timbre fiscal (1 TND par défaut) n'est appliqué **qu'une seule fois** dans les totaux de l'entête, pas par ligne.
- ☐ **Totaux entête = somme des lignes + timbre** : `invoices.amount_ht` = somme des `amount_ht` des lignes ; `invoices.vat_amount` = somme des `vat_amount` des lignes ; `invoices.daily_tax_amount` = somme des `daily_tax_amount` des lignes ; `invoices.amount_tnd` = somme($HT + TVA + \text{taxe par ligne}$) + timbre (une fois).
- ☐ **Cas limite — 1 seule ligne** : facture à 1 ligne → comportement identique à BR15bis pré-V2 (pas de régression sur les factures mono-ligne).
- ☐ **Cas limite — `nb_jours_ligne = 0` ou période d'un seul jour** : vérifier le calcul de `daily_tax_amount` pour une période de 1 jour ($nb_jours = 1 \rightarrow \text{taxe} = 1 \times \text{taxe_par_jour}$, pas 0).
- ☐ **Sens inverse TTC → HT (ré-édition)** : sur une facture existante, modifier `amount_tnd` (TTC cible) → vérifier que le système recalcule `amount_ht/vat_amount` en sens inverse, **par ligne ou globalement** selon l'implémentation retenue (à clarifier dans la spec si ambigu pour le multi-lignes).

C. Facture sans ligne interdite (BR21)

- ☐ **UI — bouton désactivé sans ligne** : ouvrir le formulaire de création de facture sans sélectionner de contrat (donc `invoice_lines` vide) et sans ajouter de ligne manuelle → le bouton "Enregistrer" est désactivé, ou un message rouge inline "Une facture doit contenir au moins une ligne." est affiché si l'utilisateur tente d'enregistrer.
- ☐ **UI — suppression de toutes les lignes** : générer des lignes via sélection de contrat, puis les supprimer une à une jusqu'à 0 → le bouton "Enregistrer" se désactive dynamiquement.
- ☐ **Backend — POST /invoices avec invoice_lines vide** : appel direct API avec `invoice_lines: []` → réponse 400 { success: false, error: "empty_invoice", message: "Une facture doit contenir au moins une ligne." }.
- ☐ **Backend — POST /rpc/create_invoice_with_lines avec lines vide** : même contrôle côté RPC → 400 empty_invoice.
- ☐ **Backend — PUT /invoices/:id** : si la mise à jour d'une facture existante peut vider ses lignes via l'API, vérifier que le même contrôle s'applique (ou clarifier si PUT ne gère pas les lignes et que la suppression passe par /invoice-lines/:id).

D. Remplacement de `invoices.lines` (JSONB) par `invoice_lines`

- ☐ **Nouvelle facture → écrite dans invoice_lines** : créer une facture → vérifier en base que les lignes sont dans la table relationnelle `invoice_lines` (avec `contract_line_id` renseigné si générées depuis un contrat), et que `invoices.lines` (JSONB) n'est plus alimenté pour les nouvelles factures (ou alimenté en parallèle si une période de transition est prévue — à clarifier).
- ☐ **Facture existante (pré-migration) avec invoices.lines JSONB uniquement** : ouvrir une facture créée avant la migration → `generateInvoicePdf` lit le repli `invoices.lines` et génère un PDF correct (pas d'erreur, pas de lignes manquantes).
- ☐ **Migration des factures existantes vers invoice_lines** : si une migration de données est prévue (copie `invoices.lines` JSONB → `invoice_lines`), vérifier qu'après migration, une facture pré-existante dispose de `invoice_lines` cohérentes avec son ancien JSONB (mêmes montants, véhicules, périodes), et que le PDF généré est identique avant/après migration.
- ☐ **GET /invoices/:id/lines** : pour une facture (nouvelle et migrée), l'endpoint renvoie la liste correcte des `invoice_lines`.

E. PDF facture (`generateInvoicePdf`)

- ☐ **PDF multi-lignes** : générer le PDF d'une facture à 3 lignes (véhicules différents) → le tableau du PDF affiche bien les 3 lignes avec véhicule/immatriculation/période/jours/HT/TVA/TTC par ligne, et les totaux + timbre fiscal une seule fois en pied de page.
- ☐ **PDF — RIB sélectionné (lien Phase 1A)** : pour une facture créée avec RIB n°2 (Phase 1A), le PDF multi-lignes affiche toujours le RIB figé correct.
- ☐ **PDF — audit non exposé** : vérifier que les champs `created_by/updated_by` (internes) ne sont **pas** affichés dans le PDF client (uniquement dans l'éditeur générique interne).

F. Atomicité (BR20bis) — `create_invoice_with_lines`

- ☐ **Création réussie en une transaction** : créer une facture à 2 lignes valides → une seule entête `invoices` + 2 `invoice_lines` créées via `POST /rpc/create_invoice_with_lines`.
- ☐ **Échec → rollback complet** : forcer un échec sur une ligne (ex. `invoice_lines` vide détecté côté RPC) → aucune entête `invoices` orpheline créée.

G. Persistance API et navigation cliquable

- ☐ **loadDataFromAPI charge invoices + invoice_lines** : recharger la page → l'écran Factures et le détail facture affichent les lignes depuis l'API.
- ☐ **syncStateToAPI//demo/reset incluent invoice_lines** : "Charger données démo" → les factures démo ont des **invoice_lines** cohérentes (pas de facture vide générée par le seed, ce qui violerait BR21 lui-même).
- ☐ **Navigation widget → facture filtrée** : depuis le dashboard, un KPI lié au chiffre d'affaires/factures impayées navigue vers **#invoices** avec le filtre de statut correspondant déjà appliqué (non-régression + cohérence avec le nouveau modèle entête+lignes).
- ☐ **Détail facture → détail contrat d'origine** : depuis le détail d'une **invoice_line** générée depuis un contrat, un lien/clic permet de naviguer vers la **contract_line** d'origine (**invoice_lines.contract_line_id** → détail du contrat, Phase 2.E).

H. Non-régression Phase 3

- ☐ **Paiements liés aux factures** : créer un paiement (**payments**) pour une facture multi-lignes → le montant dû/payé est calculé sur le total de l'entête **invoices** (somme des lignes + timbre), comportement inchangé pour l'utilisateur.
- ☐ **Tri/filtre génériques sur Factures et invoice_lines** (si Phase 1B livrée) : fonctionnels sur la nouvelle grille de lignes du détail facture.
- ☐ **Audit sur invoice_lines** : **created_by/updated_by/created_at/updated_at** renseignés et résolus si **invoice_lines** est exposé dans l'éditeur générique.
- ☐ **Écran liste Factures — colonnes inchangées** : la grille **#invoices** continue d'afficher Id, Client, Date, Statut, Total HT, Total TTC, etc. sans régression visuelle malgré le changement de modèle de données sous-jacent.

Phase 4 — Devis entête + lignes, validation et conversion (BR27, BR20bis, liens BR18/BR19/BR25)

A. Création d'un devis

- ☐ **Création devis multi-lignes** : créer un devis (**quotes**) avec un client, une date de devis, et 2 lignes (**quote_lines**) véhicule/période/tarif → calcul HT⇒TTC par ligne (BR18, comme Phase 2.B), totaux entête = somme des lignes.
- ☐ **validity_date obligatoire avec valeur par défaut** : à la création, vérifier que **validity_date** est pré-rempli à **quote_date + 30 jours**, modifiable par l'utilisateur.
- ☐ **validity_date vidé → blocage** : vider le champ "Date de validité" et tenter d'enregistrer → le formulaire bloque l'enregistrement (validation **NOT NULL** côté UI et backend).
- ☐ **validity_date < quote_date** : saisir une date de validité antérieure à la date du devis → cas limite à valider : message d'erreur ou acceptation silencieuse ? (recommandé : message d'erreur "La date de validité doit être postérieure à la date du devis.").
- ☐ **Pas de contrôle de chevauchement à la création (BR19 exclu)** : créer un devis avec une ligne sur véhicule X / période P, alors qu'une **contract_line** active existe déjà pour X sur P → **aucun blocage**, le devis est créé normalement (BR27 : un devis ne réserve pas).
- ☐ **Pas de réservation automatique à la création (BR25 exclu)** : après création du devis ci-dessus, vérifier qu'**aucune reservations** n'a été créée pour la ligne du devis.

- ☐ **Statut initial** : un devis nouvellement créé est au statut **brouillon**.

B. Écran liste et détail Devis

- ☐ **Écran liste #quotes** : colonnes Id, Client, Date du devis, Date de validité, Statut, Total HT, Total TTC, Nombre de véhicules — tri par défaut **created_at** décroissant (BR23), tri/filtre génériques fonctionnels (BR24, si Phase 1B livrée).
- ☐ **Double-clic → détail entête + lignes** : pavé haut entête (client, date, date de validité, statut, notes, totaux HT/TVA/TTC lecture seule) + pavé bas **quote_lines** (Véhicule, Période, Jours/Mois, Tarif HT, Montant HT, TVA, Montant TTC, Action "Supprimer la ligne").
- ☐ **Ajout de ligne sur devis brouillon** : "+ Ajouter une ligne" sur un devis **brouillon** → sélection véhicule/période/tarif, calcul HT⇌TTC, **sans contrôle de chevauchement** → ligne ajoutée, totaux recalculés.
- ☐ **Suppression de ligne sur devis brouillon/envoyé** : supprimer une **quote_line** → totaux recalculés.

C. Cycle de vie et expiration (statuts)

- ☐ **brouillon → envoyé** : bouton "Marquer comme envoyé" sur un devis **brouillon** → **quotes.status = envoyé**.
- ☐ **envoyé → refuse** : bouton "Refuser" → **quotes.status = refuse**, devis non convertible ensuite (vérifier que le bouton "Valider" disparaît ou se désactive pour un devis **refuse**).
- ☐ **Expiration affichée envoyé → expire** : pour un devis **envoyé** dont **validity_date < aujourd'hui**, l'écran liste affiche le statut **expire** (badge calculé à l'affichage), **sans écriture immédiate** (vérifier via API directe que **quotes.status** reste **envoyé** juste après affichage).
- ☐ **Persistance de l'expiration à la prochaine action** : effectuer une action sur ce devis (ex. "Refuser", ou tentative de "Valider") → **quotes.status** est persisté à **expire** en base à cette occasion (et si l'action était "Valider", elle doit être bloquée puisque le devis est expiré — cf. D).
- ☐ **Devis brouillon expiré** : un devis **brouillon** (jamais envoyé) dont **validity_date < aujourd'hui** — clarifier si la transition → **expire** s'applique aussi aux devis **brouillon** ou seulement **envoyé** (la spec mentionne **envoyé → expire** explicitement) ; tester le comportement réel et vérifier sa cohérence avec la spec.

D. Export PDF du devis

- ☐ **PDF devis — gabarit complet** : "Télécharger PDF" sur un devis multi-lignes → le PDF reprend le gabarit du contrat (logo/nom agence, adresse, RIB, matricule fiscal, infos client, tableau des lignes véhicule/immatriculation/période/jours/HT/TVA/TTC, totaux), **plus la date de validité affichée en évidence**.
- ☐ **PDF devis — RIB (cohérence Phase 1A)** : si le devis affiche un RIB, vérifier la cohérence avec le(s) RIB configuré(s) en paramètres (le devis n'a pas de **rib/rib_label** figés comme la facture — clarifier si le PDF devis utilise le RIB n°1 courant des paramètres, par défaut).
- ☐ **PDF devis expire** : générer le PDF d'un devis affiché **expire** → le PDF reflète le statut réel (ex. mention "Devis expiré" ou la date de validité dépassée visible), pas un statut obsolète.

E. Validation et conversion en contrat (BR27, BR20bis)

- ☐ **Validation — confirmation explicite** : sur un devis **brouillon** ou **envoyé**, cliquer "Valider" → une boîte de confirmation (`confirm()`) rappelle le caractère irréversible de l'action ; annuler la confirmation → aucune action effectuée.
- ☐ **Validation réussie — création du contrat** : confirmer la validation d'un devis à 2 lignes sans conflit → appel `POST /quotes/:id/validate` (→ `/rpc/validate_quote`) → un nouveau **contracts** (entête, statut probablement **actif**) + 2 **contract_lines** sont créés avec les mêmes valeurs (client, périodes, tarifs, HT/TVA/TTC) que les **quote_lines**.
- ☐ **quotes.converted_contract_id renseigné** : après validation réussie, **quotes.status** = **valide** et **quotes.converted_contract_id** = id du contrat créé.
- ☐ **Devis validé → lecture seule** : après validation, tenter de modifier un champ de l'entête devis ou d'éditer/supprimer une **quote_line** → toutes les actions d'édition sont désactivées ; tenter `PUT/DELETE /quotes/:id` ou `/quote-lines/:id` via API directe → rejeté (le contrôle backend empêche la modification/suppression d'un devis **valide**, cf. API_REFERENCE).
- ☐ **Devis validé → non re-convertible** : le bouton "Valider" est absent/désactivé pour un devis déjà **valide**.
- ☐ **Navigation post-validation** : après validation réussie, l'écran bascule sur le contrat créé (ou affiche un lien "Voir le contrat {id}") → cliquer ce lien ouvre bien le détail du nouveau contrat (Phase 2.E).
- ☐ **Contrôle BR19 appliqué aux lignes converties** : valider un devis dont une ligne (véhicule X / période P) chevauche désormais une **contract_line active** d'un autre contrat (créée après la création du devis) → la validation échoue, rollback complet (aucun contrat créé), message d'erreur rouge inline identique à Phase 2.C, **quotes.status** reste **envoyé/brouillon** (pas **valide**), **converted_contract_id** reste NULL.
- ☐ **Réservations créées (BR25) lors de la conversion** : pour un devis validé avec succès, vérifier que pour chaque **contract_line** créée, la séquence BR25 (Phase 2.F) s'est bien exécutée — réservations créées/liées en base.
- ☐ **Atomicité de la conversion (BR20bis)** : forcer un échec lors de la conversion (ex. conflit BR19 sur la 2e ligne d'un devis à 2 lignes) → vérifier qu'**aucun** contrat ni **contract_lines** ni **reservations** n'a été créé pour la 1ère ligne (qui aurait pu réussir isolément) — rollback complet de **validate_quote**.
- ☐ **Validation d'un devis refuse ou expire** : le bouton "Valider" est absent/désactivé pour un devis **refuse** ou **expire** ; tentative via API directe (`POST /quotes/:id/validate`) sur un devis **refuse/expire** → rejetée avec message explicite.

F. Persistance API (checklist transversale)

- ☐ **loadDataFromAPI charge quotes + quote_lines** : recharger la page → écran Devis affiche les entêtes et lignes depuis l'API.
- ☐ **Écriture via formulaires** : création/édition de devis et lignes persistées via `POST/PUT /quotes`, `/quote-lines` (et `/rpc/validate_quote` pour la conversion), avec mise à jour de **state.quotes/state.quoteLines** après succès.
- ☐ **syncStateToAPI//demo/reset incluent quotes/quote_lines** : "Charger données démo" → le jeu de données démo contient des devis dans différents statuts (**brouillon**, **envoyé**, **valide** avec contrat lié, **expire**) pour permettre de tester tous les cas ci-dessus sans recréer manuellement les données.

G. Navigation cliquable (widgets Devis)

- ☐ **KPI "Devis en attente" (dashboard)** : cliquer sur ce KPI → navigue vers **#quotes** filtré **status=envoyé** (filtre visible dans la ligne de filtres générique si Phase 1B livrée).

- ☐ **KPI "Devis expirant bientôt"** : cliquer → navigue vers `#quotes` filtré `status=envoye`, trié par `validity_date` croissant (vérifier que le tri appliqué est bien visible/actif dans l'en-tête de colonne).
- ☐ **Lien devis validé → contrat** (couvre aussi E) : depuis l'écran liste Devis, une ligne `valide` affiche un lien/bouton vers `converted_contract_id` → navigation vers le détail du contrat correspondant.

H. Non-régression Phase 4

- ☐ **Dashboard global** : les nouveaux KPI Devis n'altèrent pas l'affichage/le calcul des KPI existants (Contrats actifs, CA, etc.).
- ☐ **Écrans Contrats et Factures** : un contrat créé par conversion de devis (E) apparaît normalement dans `#contracts`, peut être facturé normalement (Phase 3), résilié (Phase 2.G), etc. — aucun traitement spécial requis ni régression liée à son origine "devis".
- ☐ **Audit et tri/filtre sur quotes/quote_lines** : `created_by/updated_by/created_at/updated_at` renseignés et résolus (BR23) ; tri par défaut `created_at` décroissant et tri/filtre génériques fonctionnels (BR24) sur l'écran Devis.
- ☐ **Suppression d'un devis brouillon/envoye/refuse/expire non converti** : `DELETE /quotes/:id` sur un devis non `valide` → suppression complète de l'entête et de ses `quote_lines` (cascade), sans effet sur `contracts/reservations`.

Points d'attention transverses à vérifier en fin de chaque phase

- ☐ **Aucun écran local-only** : pour chaque nouvelle entité de la phase, vérifier la checklist persistance (chargement `loadDataFromAPI`, écriture via formulaires, inclusion dans `syncStateToAPI//demo/reset`) — cf. `docs/04-features/FEATURE_SPECIFICATIONS.md` section 9.0.
- ☐ **Navigation cliquable** : pour chaque nouveau KPI/liste/graphique, vérifier qu'il navigue vers l'écran détaillé avec le même filtre déjà appliqué — cf. tableau section 9.0.
- ☐ **Documentation à jour** : vérifier que `docs/01-specifications/BMAD.md`, `docs/03-data-model/SCHEMA_REFERENCE.md`, `docs/04-features/FEATURE_SPECIFICATIONS.md`, `docs/05-api/API_REFERENCE.md` reflètent l'état réellement livré de la phase (pas seulement la spec cible), notamment si des écarts/ajustements ont été faits pendant le développement.
- ☐ **Messages d'erreur en français, explicites** : tous les messages de blocage (conflit BR19, facture vide BR21, devis non valide, validité obligatoire BR27) sont en français, compréhensibles par un utilisateur non technique, sans jargon technique (codes HTTP, noms de tables).
- ☐ **Comportement si API indisponible** : pour chaque nouvel écran/action de la phase, couper temporairement le backend (ou simuler une erreur réseau) → l'UI affiche un message d'erreur explicite (pas un écran blanc, pas de blocage silencieux), cohérent avec la règle "aucun écran local-only".